

2. Circuitos Combinacionales

Al hablar de sistemas, nos referimos al enfoque sistémico con el que serán tratadas las funciones de conmutación.

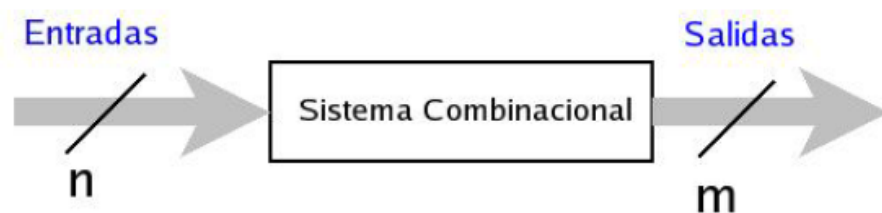
Dentro de este enfoque sistémico, existen 2 grandes áreas: los **Sistemas Combinacionales** y los **Sistemas Secuenciales**.

Los *sistemas combinacionales* están formados por un conjunto de compuertas interconectadas cuya salida, en un momento dado, esta únicamente en función de la entrada, en ese mismo instante. Por esto se dice que los sistemas combinacionales **no cuentan con memoria**.

Los *sistemas secuenciales* en cambio, son capaces de tener salidas no sólo en función de las entradas actuales, sino que también de entradas o salidas anteriores.

Esto se debe a que los sistemas secuenciales tienen memoria y son capaces de almacenar información a través de sus estados internos.

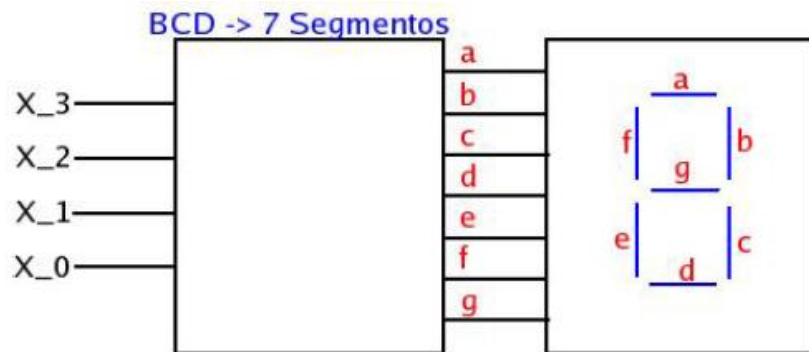
Un **sistema combinacional** puede tener n entradas y m salidas.



Un sistema secuencial puede ser visto como una “*caja negra*”, en cuyo interior hay compuertas lógicas, que representan una ecuación de conmutación.

Las condiciones superfluas corresponden a aquellos casos en que las combinaciones de variables de entrada no pueden ocurrir.

Por ejemplo, si se quiere construir un circuito combinacional para convertir números que están en BCD (de 4 bits), a siete salidas que representan los segmentos de un display.



Nos enfocaremos en el segmento inferior derecho del display (segmento c), cuya **Tabla de Verdad** corresponde a:

X_3	X_2	X_1	X_0	c	X_3	X_2	X_1	X_0	c
0	0	0	0	1	1	0	0	0	1
0	0	0	1	1	1	0	0	1	1
0	0	1	0	0	1	0	1	0	-
0	0	1	1	1	1	0	1	1	-
0	1	0	0	1	1	1	0	0	-
0	1	0	1	1	1	1	0	1	-
0	1	1	0	1	1	1	1	0	-
0	1	1	1	1	1	1	1	1	-

Se puede observar que las entradas mayores a 9 no son posibles, debido a que el código BCD solo llega hasta el 9.

Por esto, las combinaciones de entrada posteriores a 1001 no son posibles y se consideran superfluas.

Luego si construimos el MK de esta función, podemos dejar las celdas superfluas con un “-”.

$X_3 \backslash X_2 \backslash X_1 X_0$	00	01	11	10
00	1	1	-	1
01	1	1	-	1
11	1	1	-	-
10	0	1	-	-

Las celdas superfluas pueden ser consideradas como ceros o bien como unos, independientemente.

De esta manera se agrupa según conveniencia, para obtener la menor cantidad de subcubos, y que estos sean del mayor tamaño posible.

$X_3 \backslash X_2 \backslash X_1 X_0$	00	01	11	10
00	1	1	-	1
01	1	1	-	1
11	1	1	-	-
10	0	1	-	-

Resultando la ecuación:

$$F(X_3, X_2, X_1, X_0) = \overline{X_1} + X_0 + X_2$$

Los sistemas combinacionales relativamente pequeños (menores a 100 compuertas), se pueden ser construidos con compuertas convencionales.

Típicamente se utilizan únicamente compuertas NAND o NOR.

Utilizando compuertas NAND

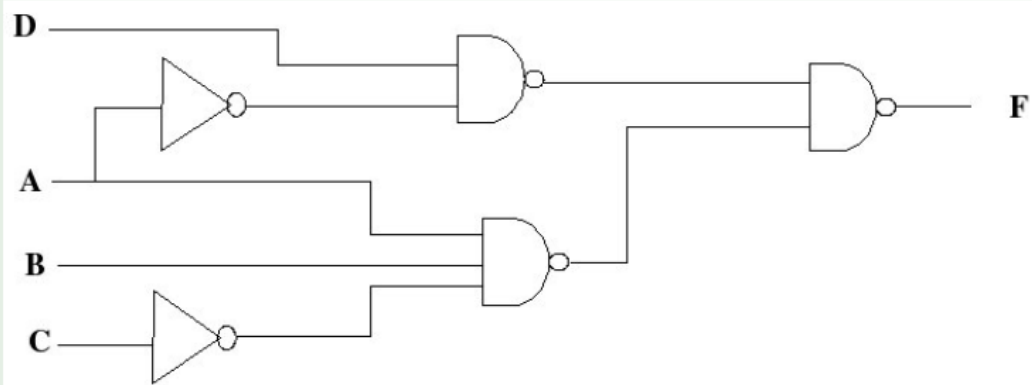
Por ejemplo, para representar la ecuación:

$$F(A, B, C, D) = \overline{A} \cdot D + B \cdot A \cdot \overline{C}$$

Algebraicamente se puede convertir:

$$\begin{aligned} F(A, B, C, D) &= \overline{A} \cdot D + B \cdot A \cdot \overline{C} \\ &= \overline{(\overline{A} \cdot D) \cdot (B \cdot A \cdot \overline{C})} \end{aligned}$$

Utilizando compuertas NAND



Utilizando compuertas NOR

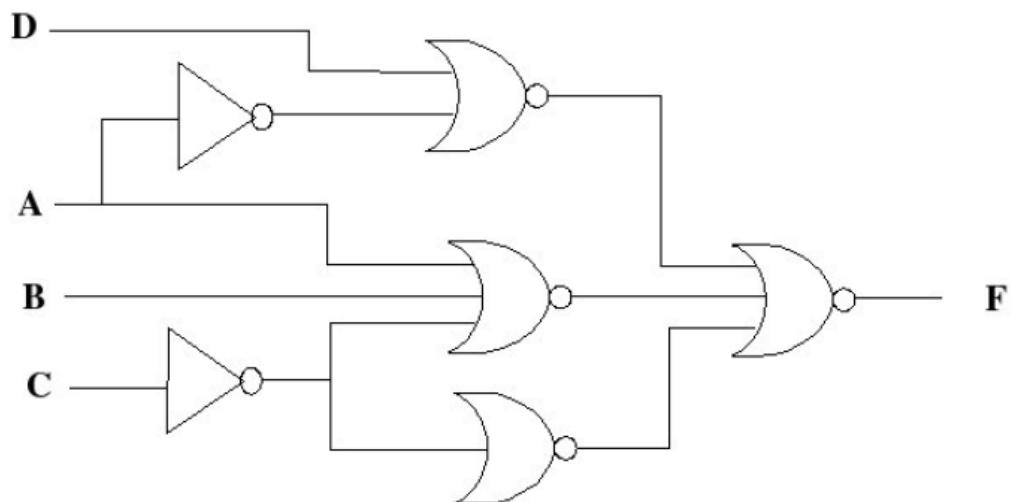
Representar la siguiente ecuación utilizando compuertas NOR:

$$F(A, B, C, D) = (\bar{A} + D) \cdot (B + A + \bar{C}) \cdot C$$

Algebraicamente se puede convertir:

$$\begin{aligned}
 F(A, B, C, D) &= \frac{(\bar{A} + D) \cdot (B + A + \bar{C}) \cdot C}{(\bar{A} + D) + (B + A + \bar{C}) + \bar{C}} \\
 &= \overline{(\bar{A} + D) + (B + A + \bar{C}) + \bar{C}}
 \end{aligned}$$

Utilizando compuertas NOR



Hasta el momento solo hemos visto chips con compuertas lógicas elementales, con las cuales es posible representar ecuaciones de conmutación.

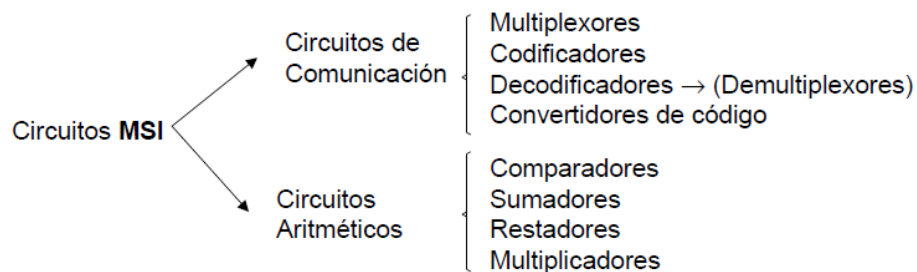
A medida que aumenta la cantidad de compuertas, nos vemos en la necesidad de construir dispositivos lógicos altamente integrados (VLSI).

Los dispositivos *VLSI* consideran una disminución en el tamaño (físico) final de la solución, en el costo por densidad de compuertas y en la latencia del circuito combinacional (debido a que las interconexiones internas son más rápidas) .

Sin embargo es necesario construir un chip distinto, según sea la aplicación, por lo que los costos en diseño son bastante altos.

Tipos de Circuitos Combinacionales según escala de integración:

- **SSI** → máx. 10 puertas lógicas (100 xtores)*
- **MSI** → máx. 100 puertas lógicas (1000 xtores)**
- **LSI** → máx. 1000 puertas lógicas (10000 xtores)
- **VLSI** → > 1000 puertas lógicas (>10000 xtores)



Los circuitos combinacionales realizados con puertas lógicas implementan funciones booleanas, pero no son los únicos elementos capaces de ello. En este tema veremos que los llamados *módulos combinacionales* pueden implementar funciones booleanas y cumplir otras misiones específicas más.

Se tratarán como bloques funcionales que realizan una función determinada, si bien se comprobará cómo es posible realizar su función específica por medio de puertas lógicas.

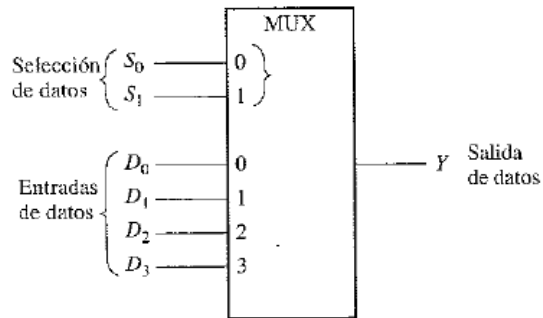
Las siglas **MSI** (**M**edium **S**ize of **I**ntegration) se refieren a aquellos circuitos de Media Escala de Integración con un número de puertas entre 10 y 100.

Por tanto, primeramente veremos los módulos combinacionales como bloques funcionales, posteriormente comprobaremos que un conjunto de puertas lógicas debidamente conectadas realizan la función del módulo, y, por último, estudiaremos cómo se pueden implementar funciones booleanas usando módulos combinacionales.

Los módulos combinacionales disponen además de las llamadas *señales de control*, que permiten que el circuito funcione normalmente, si están activadas, o permanezca inactivo si la señales de control están desactivadas.

MULTIPLEXORES (SELECTORES DE DATOS)

Un multiplexor (MUX) es un dispositivo que permite dirigir la información digital procedente de diversas fuentes a una única línea para ser transmitida a través de dicha línea a un destino común. El multiplexor básico posee varias líneas de entrada de datos y una única línea de salida. También posee entradas de selección de datos, que permiten conmutar los datos digitales provenientes de cualquier entrada hacia la línea de salida. A los multiplexores también se les conoce como selectores de datos.



Entradas de selección de datos		Entrada seleccionada
S_1	S_0	
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

Cuando sumamos lógicamente (operación OR) estos términos, la expresión total para los datos de salida será:

$$Y = D_0 \bar{S}_1 \bar{S}_0 + D_1 \bar{S}_1 S_0 + D_2 S_1 \bar{S}_0 + D_3 S_1 S_0$$

La implementación de esta ecuación requiere cuatro puertas AND de tres entradas, una puerta OR de cuatro entradas y dos inversores para generar los complementos de S_1 y S_0 , como se muestra en la Figura 6.46. Dado que los datos pueden ser seleccionados desde cualquier línea de entrada, se conoce también a este circuito con el nombre de **selector de datos**.

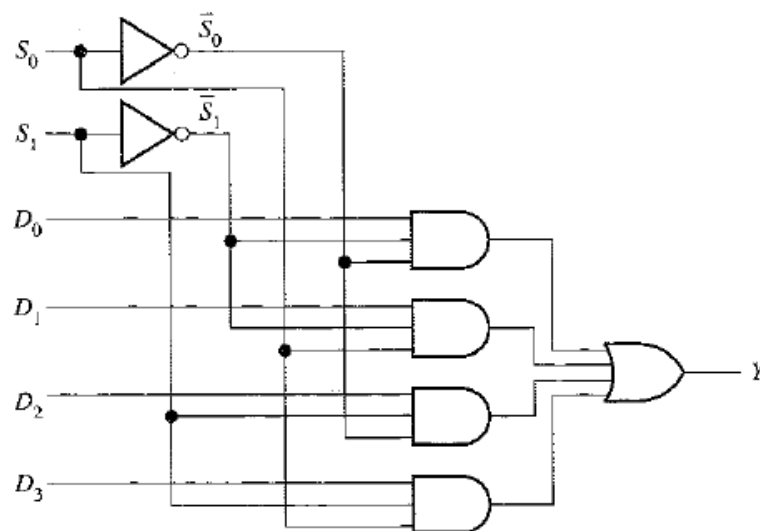
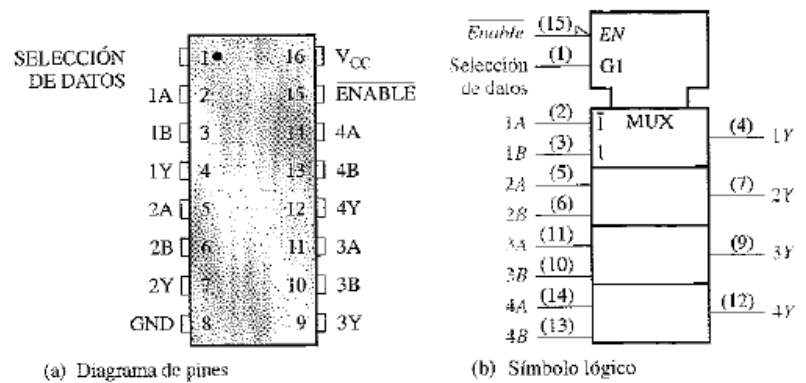


Diagrama de pines y símbolo lógico para el cuádruple selector de datos/multiplexor de dos entradas 74HC157A.



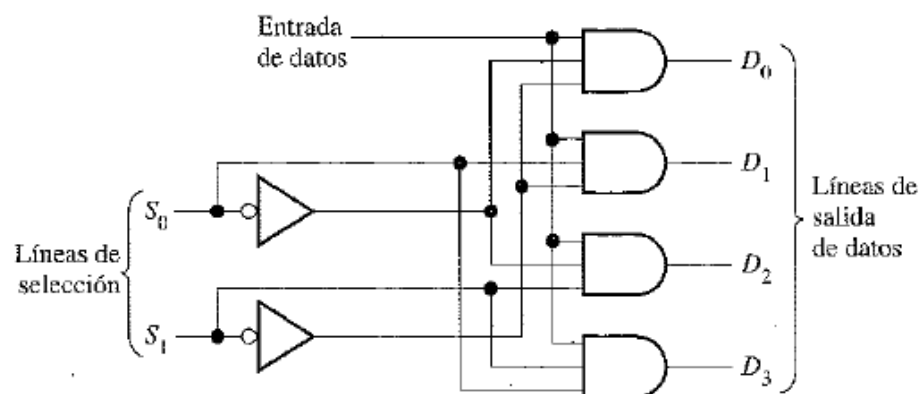
DEMÚLTIPLEXORES

Un demultiplexor (DEMUX) básicamente realiza la función contraria a la del multiplexor. Toma datos de una línea y los distribuye a un determinado número de líneas de salida. Por este motivo, el demultiplexor se conoce también como distribuidor de datos. Como veremos, los decodificadores pueden utilizarse también como demultiplexores.

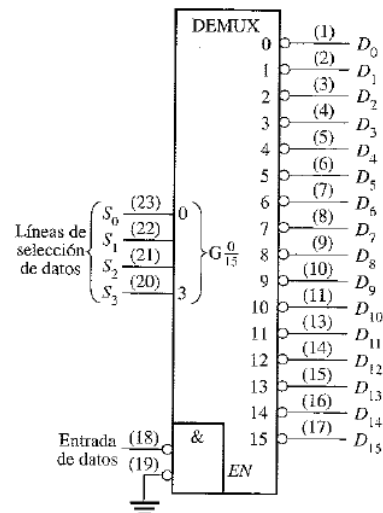
Al finalizar esta sección, el lector deberá ser capaz de:

- Explicar el funcionamiento básico de un demultiplexor.
- Describir cómo el decodificador de 4 líneas a 16 líneas 74HC154 puede utilizarse como demultiplexor.
- Desarrollar el diagrama de tiempos de un demultiplexor con un número determinado de líneas de entrada y de líneas de selección de datos.

La Figura 6.54 muestra un circuito demultiplexor (DEMUX) de 1 línea a 4 líneas. La línea de entrada de datos está conectada a todas las puertas AND. Las dos líneas de selección de datos activan únicamente una puerta cada vez y los datos que aparecen en la línea de entrada de datos pasarán a través de la puerta seleccionada hasta la línea de salida de datos asociada.



El 74HC154 utilizado
como demultiplexor.

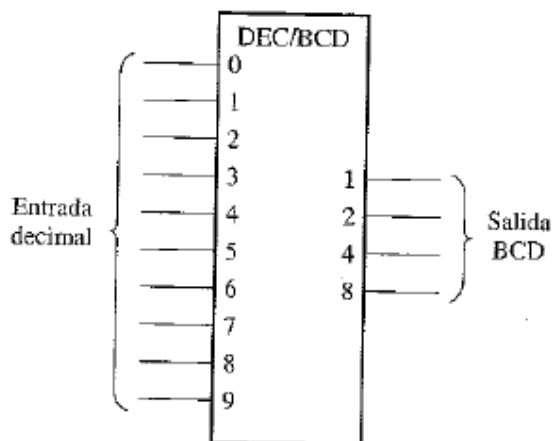


CODIFICADORES

Un codificador es un circuito lógico combinacional que, esencialmente, realiza la función "inversa" del decodificador. Un codificador permite que se introduzca en una de sus entradas un nivel activo que representa un dígito, como puede ser un dígito decimal u octal, y lo convierte en una salida codificada, como BCD o binario. Los codificadores se pueden diseñar también para codificar símbolos diversos y caracteres alfabéticos. El proceso de conversión de símbolos comunes o números a un formato codificado recibe el nombre de codificación.

Codificador decimal-BCD

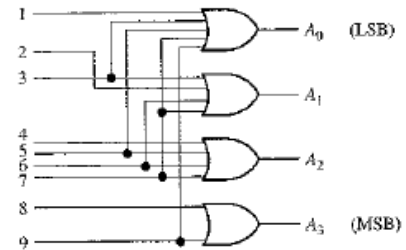
Este tipo de codificador posee diez entradas, una para cada dígito decimal, y cuatro salidas que corresponden al código BCD, como se muestra en la Figura 6.33. Este es un codificador básico de 10 líneas a 4 líneas.



Dígito decimal	Código BCD			
	A ₃	A ₂	A ₁	A ₀
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

El funcionamiento básico del circuito de la Figura 6.34 es el siguiente: cuando aparece un nivel ALTO en una de las líneas de entrada correspondientes a los dígitos decimales, se generan los niveles apropiados en las cuatro líneas BCD de salida. Por ejemplo, si la línea de entrada 9 está a nivel ALTO (suponiendo que todas las demás entradas están a nivel BAJO), esta condición producirá un nivel ALTO en las salidas A_0 y A_3 , y un nivel BAJO en A_1 y A_2 , que es el código BCD (1001) del número decimal 9.

FIGURA 6.34
Diagrama lógico básico de un codificador decimal-BCD. No se necesita una entrada para el dígito 0, ya que las salidas BCD están todas a nivel BAJO cuando no hay entradas a nivel ALTO.



DECODIFICADORES

La función básica de un decodificador es detectar la presencia de una determinada combinación de bits (código) en sus entradas y señalar la presencia de este código mediante un cierto nivel de salida. En su forma más general, un decodificador posee, líneas de entrada para gestionar n bits, y en una de las 2^n líneas de salida indica la presencia de una o más combinaciones de n bits. En esta sección, se presentarán varios tipos de decodificadores. Los principios básicos se pueden extender a otros tipos de decodificadores.

El decodificador binario básico

Supongamos que necesitamos determinar cuándo aparece el número binario 1001 en las entradas de un circuito digital. Se puede utilizar una puerta AND como elemento básico de decodificación, ya que produce una salida a nivel ALTO sólo cuando todas sus entradas están a nivel ALTO. Por tanto, debe asegurarse de que todas las entradas de la puerta AND estén a nivel ALTO cuando se introduce el número 1001, lo cual se puede conseguir invirtiendo las dos entradas centrales (cuyos bits son 0), como se muestra en la Figura 6.22.

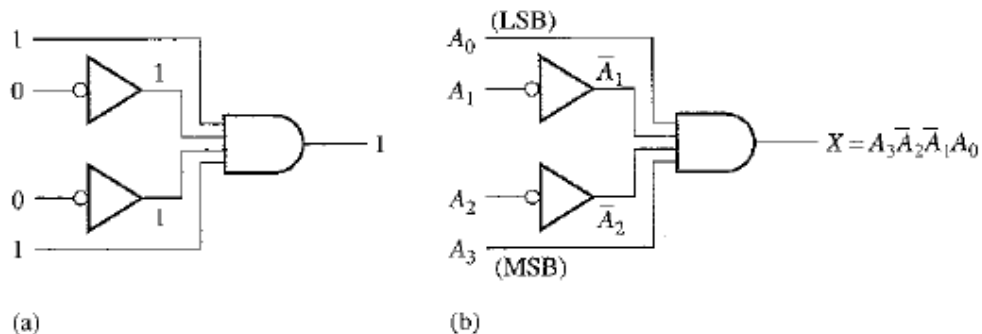


FIGURA 6.22
Lógica de decodificación del código binario 1001 con una salida activa a nivel ALTO.

La ecuación lógica para el **decodificador** de la Figura 6.22(a) se desarrolla como se ilustra en la Figura 6.22(b). Se debe comprobar que la salida es siempre 0 excepto cuando se aplican las entradas $A_0 = 1$, $A_1 = 0$, $A_2 = 0$ y $A_3 = 1$. A_0 es el bit menos significativo y A_3 el más significativo. *En este libro, cuando se representa un número binario o cualquier otro código de pesos, el bit menos significativo siempre es el situado más a la derecha cuando el número se escribe en sentido horizontal, y el de más arriba cuando se escribe en vertical, a menos que se indique lo contrario.*

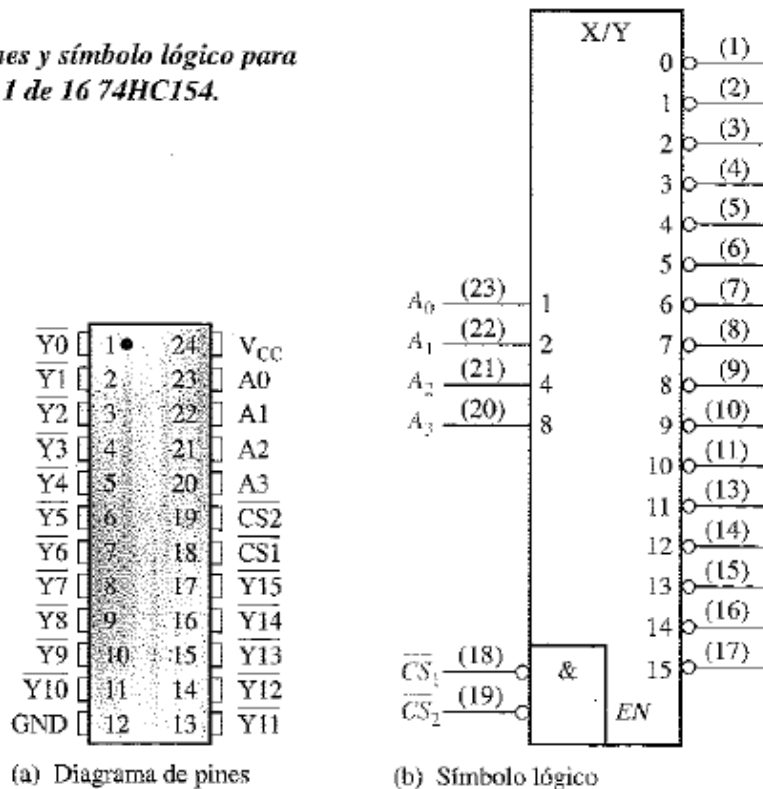
Si se utiliza una puerta NAND en lugar de una AND, como se muestra en la Figura 6.22, una salida a nivel BAJO indicará la presencia del código binario adecuado, que en este caso es 1001.

Un decodificador MSI 1 de 16

El 74HC154 es un buen ejemplo de un decodificador MSI. Su símbolo lógico se muestra en la Figura 6.25. En este tipo de dispositivo existe una función de activación (enable, *EN*), que se implementa mediante una puerta NOR utilizada como negativa-AND. En las entradas de selección del chip, $\overline{CS}_1$ y $\overline{CS}_2$, se requiere un nivel BAJO para obtener en la salida de la puerta de activación (*EN*, enable) un nivel ALTO. La salida de la puerta de activación se conecta a una entrada de *cada* puerta NAND del decodificador, por lo que debe estar a nivel ALTO para que las puertas NAND se activen. Si la puerta de activación no se activa mediante un nivel BAJO en ambas entradas, entonces las 16 salidas (*Y*) del decodificador estarán a nivel ALTO independientemente del estado de las cuatro variables de entrada A_0 , A_1 , A_2 y A_3 .

FIGURA 6.25

Diagrama de pines y símbolo lógico para el decodificador 1 de 16 74HC154.



El decodificador BCD a decimal

Un decodificador BCD a decimal convierte cada código BCD (código 8421) en uno de los diez posibles dígitos decimales. Frecuentemente, se le denomina *decodificador de 4 líneas a 10 líneas* o *decodificador 1 de 10*.

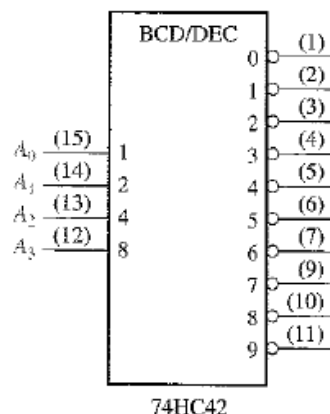
El método de implementación es el mismo que hemos visto anteriormente para el decodificador de 4 líneas a 16 líneas, excepto que ahora sólo se requieren 10 puertas decodificadoras, dado que el código BCD sólo representa los diez dígitos decimales de 0 a 9. En la Tabla 6.5 se muestra una lista de los diez códigos BCD y sus correspondiente funciones de decodificación. Cada una de estas funciones se implementa mediante puertas NAND para proporcionar salidas activas a nivel BAJO. Si se requirieran salidas activas a nivel ALTO, se utilizarían puertas AND para la decodificación. La lógica es idéntica a la de las diez primeras puertas del decodificador de 4-líneas a 16 líneas (véase la Tabla 6.4)

TABLA 6.5
Funciones de decodificación BCD.

Dígito decimal	Código BCD				Función de decodificación
	A_3	A_2	A_1	A_0	
0	0	0	0	0	$\overline{A_3}\overline{A_2}\overline{A_1}\overline{A_0}$
1	0	0	0	1	$\overline{A_3}\overline{A_2}\overline{A_1}A_0$
2	0	0	1	0	$\overline{A_3}\overline{A_2}A_1\overline{A_0}$
3	0	0	1	1	$\overline{A_3}\overline{A_2}A_1A_0$
4	0	1	0	0	$\overline{A_3}A_2\overline{A_1}\overline{A_0}$
5	0	1	0	1	$\overline{A_3}A_2\overline{A_1}A_0$
6	0	1	1	0	$\overline{A_3}A_2A_1\overline{A_0}$
7	0	1	1	1	$\overline{A_3}A_2A_1A_0$
8	1	0	0	0	$A_3\overline{A_2}\overline{A_1}\overline{A_0}$
9	1	0	0	1	$A_3\overline{A_2}\overline{A_1}A_0$

El 74HC42 es un circuito integrado decodificador BCD-decimal. Su símbolo lógico se muestra en la Figura 6.28. Dibujar las señales de salida si se aplican las formas de onda de entrada de la Figura 6.29(a) a las entradas del 74HC42.

FIGURA 6.28
El decodificador BCD a decimal 74HC42.



Solución. Las formas de onda de salida se muestran en la Figura 6.29(b). Como puede verse, las entradas al decodificador constituyen una secuencia de dígitos de 0 a 9. Las formas de onda de salida del cronograma indican dicha secuencia.

Problema relacionado. Construir un diagrama de tiempos que muestre las señales de entrada y de salida para el caso en que la secuencia binaria de entrada origine los siguientes números decimales: 0, 2, 4, 6, 8, 1, 3, 5 y 9.

Un codificador MSI 8 líneas a 3 líneas (octal-binario)

El 74F148 es un codificador con prioridad que tiene ocho entradas activas a nivel BAJO y tres salidas binarias activas a nivel BAJO, como se muestra en la Figura 6.36. Este dispositivo se puede utilizar para convertir entradas octales (recuérdese que los dígitos octales son del 0 al 7) en código binario de 3 bits. Para activar este dispositivo, la entrada de activación (enable input, *EI*) tiene que estar activa a nivel BAJO. También tiene una *EO* (salida de activación, enable output) y una salida *GS* para permitir la ampliación. La salida *EO* está a nivel BAJO cuando la entrada *EI* está a nivel BAJO y ninguna de las entradas (de 0 a 7) se encuentra activada. *GS* está a nivel BAJO cuando *EI* está a nivel BAJO y cualquiera de las entradas se encuentra activada.

El 74F148 puede ser ampliado a un codificador de 16 líneas a 4 líneas conectando la salida *EO* del codificador de mayor orden a la entrada *EI* del codificador de menor orden, y aplicando la operación negativa-OR a las correspondientes salidas binarias, como se muestra en la Figura 6.37. La salida *EO* se utiliza como cuarto y más significativo bit. Esta configuración particular produce salidas activas a nivel ALTO para los números binarios de cuatro bits.

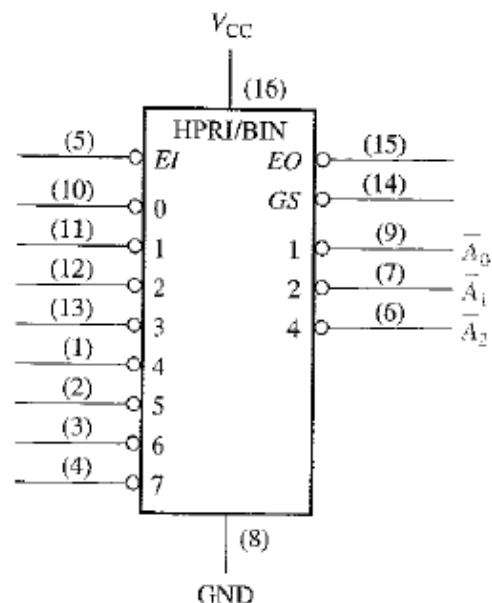


FIGURA 6.36
Símbolo lógico del codificador 8 líneas a 3 líneas 74F148.

CONVERTIDORES DE CÓDIGO

En esta sección, vamos a examinar algunos métodos que utilizan circuitos lógicos combinacionales para pasar de un código a otro.

Conversión BCD-binario

Uno de los métodos de conversión de código BCD a binario utiliza circuitos sumadores. El proceso básico de conversión consiste en lo siguiente:

1. El valor, o peso, de cada uno de los bits de un número BCD se representa por un número binario.
2. Se suman todas las representaciones binarias de los pesos de los bits del número BCD que son 1.
3. El resultado de la suma es el equivalente binario del número BCD.

Una manera más concisa de expresar esta operación es:

Para obtener el número binario completo hay que sumar los números binarios que representan los pesos de los bits del número BCD.

Vamos a examinar un código BCD de 8 bits (que representa un número decimal de 2 dígitos) para comprender la relación entre el código binario y el BCD. Por ejemplo, ya sabemos que el número decimal 87 se puede expresar en BCD como:

$$\begin{array}{cc} \underbrace{1000} & \underbrace{0111} \\ 8 & 7 \end{array}$$

El grupo de 4 bits más a la izquierda representa 80 y el grupo de 4 bits más a la derecha representa el número 7. Esto quiere decir: el grupo más a la izquierda tiene un peso de 10 y el de más a la derecha tiene un peso de 1. Dentro de cada grupo, el peso binario de cada bit es el siguiente:

	Dígito de las decenas				Dígito de las unidades			
Peso:	80	40	20	10	8	4	2	1
Designación de bit:	B_3	B_2	B_1	B_0	A_3	A_2	A_1	A_0

El equivalente binario de cada bit BCD es un número binario que representa el peso de cada bit dentro del número BCD completo. Esta representación se muestra en la Tabla 6.7.

TABLA 6.7
Representación
binaria de los
pesos de los bits
BCD.

Bit BCD	Peso BCD	Representación binaria						
		(MSB)						(LSB)
		64	32	16	8	4	2	1
A_0	1	0	0	0	0	0	0	1
A_1	2	0	0	0	0	0	1	0
A_2	4	0	0	0	0	1	0	0
A_3	8	0	0	0	1	0	0	0
B_0	10	0	0	0	1	0	1	0
B_1	20	0	0	1	0	1	0	0
B_2	40	0	1	0	1	0	0	0
B_3	80	1	0	1	0	0	0	0

Convertidores MSI BCD-binario y binario-BCD

Los **convertidores de código MSI** se suelen implementar con dispositivos lógicos programables (PLD, programmable logic device), que se presentarán en el Capítulo 7. Ejemplos de estos dispositivos son el 74184, un dispositivo PLD preprogramado como convertidor BCD-binario, y el 74185, dispositivo PLD preprogramado como un convertidor binario-BCD. Los símbolos lógicos de estos dispositivos utilizados como convertidores de 6 bits se muestran en la Figura 6.39. Las Figuras 6.40 y 6.41 muestran cómo estos dispositivos pueden ampliarse para convertir números con mayor cantidad de bits.

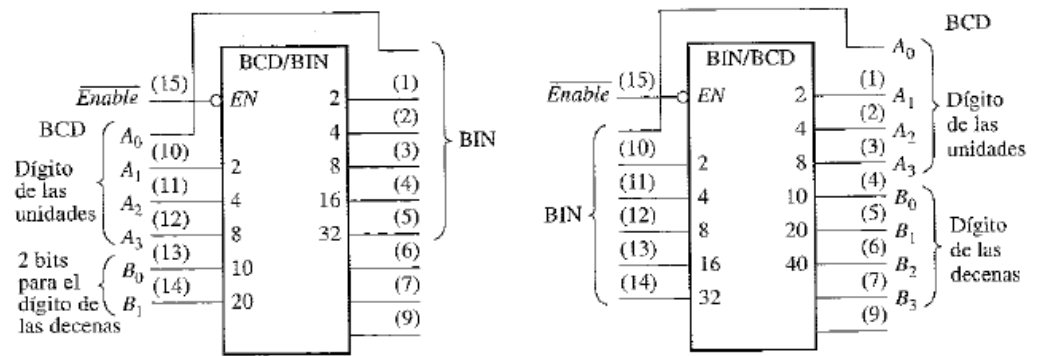
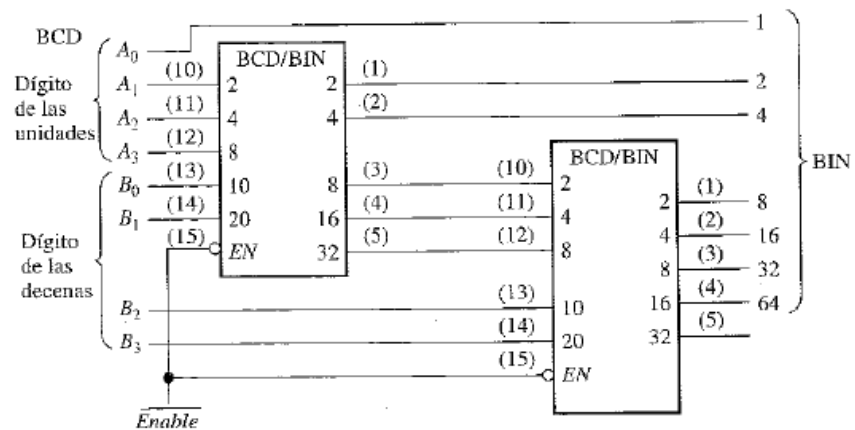


FIGURA 6.39

(a) Convertidor BCD-binario de 6 bits 74184

(b) Convertidor BCD-binario de 6 bits 74185

 FIGURA 6.40
 Dispositivos 74184
 para la conversión
 de dos dígitos BCD
 a binario.


SUMADORES BÁSICOS

Los sumadores son muy importantes no solamente en las computadoras, sino en muchos tipos de sistemas digitales en los que se procesan datos numéricos. Comprender el funcionamiento de un sumador es fundamental en el estudio de los sistemas digitales. En esta sección estudiaremos el semisumador y el sumador completo.

El semisumador

Recordemos las reglas básicas de la suma binaria expuestas en el Capítulo 2:

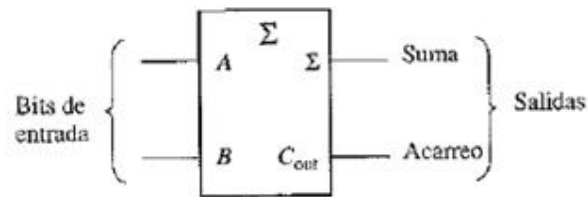
$$\begin{array}{l} 0 + 0 = 0 \\ 0 + 1 = 1 \\ 1 + 0 = 1 \\ 1 + 1 = 10 \end{array}$$

Todas estas operaciones se realizan mediante un circuito lógico denominado **semisumador**.

Un semisumador admite dos dígitos binarios en sus entradas y genera dos dígitos binarios en sus salidas: un bit de suma y un bit de acarreo.

Los semisumadores se representan mediante el símbolo lógico de la Figura 6.1.

Los semisumadores se representan mediante el símbolo lógico de la Figura 6.1.



Lógica del semisumador. A partir del funcionamiento lógico de un semisumador, expuesto en la Tabla 6.1, las expresiones correspondientes a la suma y al acarreo de salida se pueden obtener como funciones de las entradas. Obsérvese que la salida de acarreo (C_{out}) es 1 sólo cuando A y B son 1; por tanto, C_{out} puede expresarse como una operación AND de las variables de entrada.

$$C_{out} = AB \quad (6.1)$$

Obsérvese ahora que la salida correspondiente a la suma (Σ) es 1 sólo si las variables A y B son distintas. Por tanto, la suma puede expresarse como una operación OR-exclusiva de las variables de entrada.

$$\Sigma = A \oplus B \quad (6.2)$$

TABLA 6.1
Tabla de verdad de un semisumador.

A	B	C_{out}	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

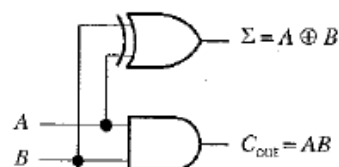
Σ = suma

C_{out} = acarreo de salida

A y B = variables de entrada (operandos)

A partir de las ecuaciones (6.1) y (6.2), se puede desarrollar la implementación lógica del funcionamiento de un semisumador. La salida de acarreo se produce mediante una puerta AND, siendo A y B sus dos entradas, y la salida de la suma se obtiene mediante una puerta OR-exclusiva, como muestra la Figura 6.2.

FIGURA 6.2
Diagrama lógico de un semisumador.



El sumador completo

El segundo tipo de sumador es el **sumador completo**.

Un sumador acepta dos bits de entrada y un acarreo de entrada, y genera una salida de suma y un acarreo de salida.

La diferencia principal entre un sumador completo y un semisumador es que el sumador completo acepta un acarreo de entrada. El símbolo lógico de un sumador completo se muestra en la Figura 6.3, y la tabla de verdad mostrada en la Tabla 6.2 describe su funcionamiento.

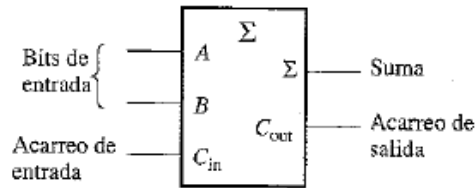


TABLA 6.2
Tabla de verdad de un sumador completo.

A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

C_{in} = acarreo de entrada, algunas veces se designa por CI

C_{out} = acarreo de salida, algunas veces se designa por CO

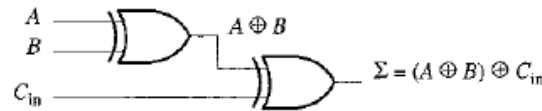
Σ = suma

A y B = variables de entrada (operandos)

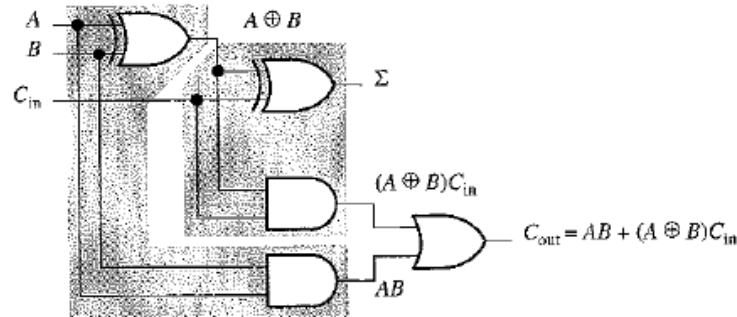
Lógica del sumador completo. Un sumador completo suma los dos bits de entrada y el bit de acarreo de entrada. A partir del semisumador, ya conocemos que la suma de los dos bits de entrada A y B consiste en la operación OR-exclusiva entre estas dos variables, $A \oplus B$. Para sumar el acarreo de entrada (C_{in}) a los bits de entrada, hay que volver a aplicar la operación OR-exclusiva, obteniéndose la siguiente ecuación de salida para el sumador completo:

$$\Sigma = (A \oplus B) \oplus C_{in} \quad (6.3)$$

Esto significa que, para implementar la función de un sumador completo, se pueden utilizar dos puertas OR-exclusiva. La primera tiene que generar el término $A \oplus B$, y la segunda toma como entradas la salida de la primera puerta XOR y el acarreo de entrada, como se muestra en la Figura 6.4(a).



(a) Lógica necesaria para realizar la suma de tres bits

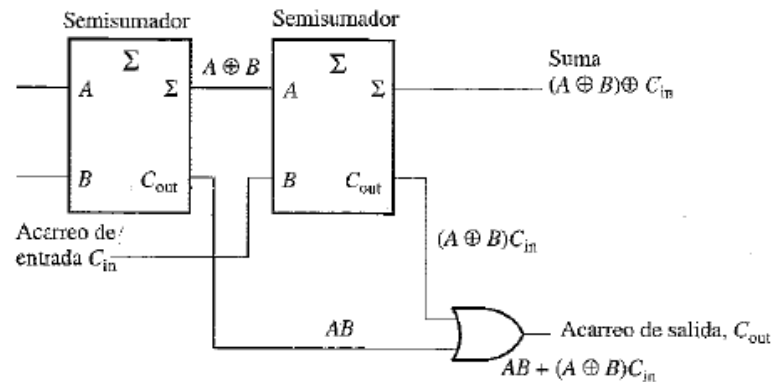


(b) Circuito lógico de un sumador completo (cada semisumador se representa por un área sombreada)

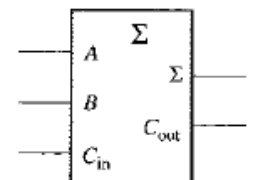
El acarreo de salida es 1 cuando las dos entradas de la primera puerta XOR son 1, o cuando las dos entradas de la segunda puerta XOR son 1. Esto se puede comprobar analizando la Tabla 6.2. El acarreo de salida del sumador completo se obtiene a partir del producto lógico (AND) de las entradas A y B , y del producto lógico (AND) de $A \oplus B$ y de C_{in} , sumando (OR) después ambos términos resultantes, como se muestra en la ecuación (6.4). Esta función, una vez implementada, se combina con la de la suma lógica para constituir un circuito sumador completo, como se muestra en la Figura 6.4(b).

$$C_{out} = AB + (A \oplus B)C_{in} \quad (6.4)$$

Obsérvese que, en la Figura 6.4(b), existen dos semisumadores conectados, como se muestra en el diagrama de bloques de la Figura 6.5(a), cuyos acarreos de salida se aplican a una puerta OR. El símbolo lógico mostrado en la Figura 6.5(b) será el que normalmente empleemos para representar un sumador completo.



(a) Dos semisumadores formando un sumador completo

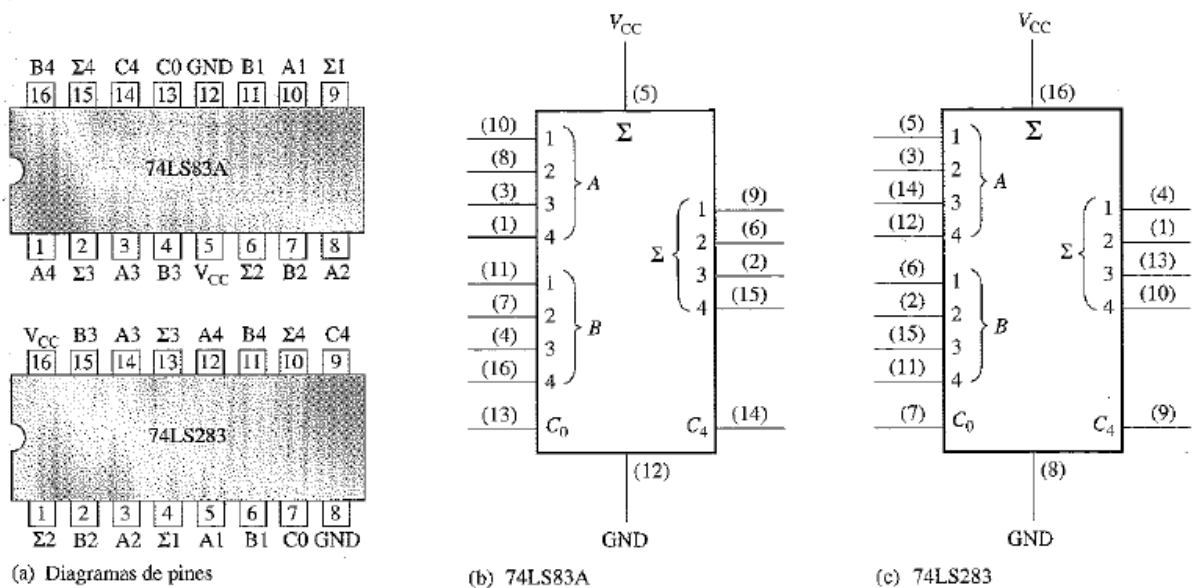


(b) Símbolo lógico de un sumador completo

Sumadores MSI

Ejemplos de sumadores paralelo de 4 bits que están disponibles como circuitos integrados de media escala (MSI) son los dispositivos TTL Schottky de baja potencia 74LS83A y 74LS283. Estos dos dispositivos son funcionalmente idénticos entre sí, aunque no son compatibles en cuanto a la disposición de sus pines; es decir, los números de pin para las entradas y salidas son diferentes debido a que las conexiones de los pines de masa y alimentación son distintos. Para el 74LS83A, V_{CC} es el pin 5 y tierra es el pin 12 en el encapsulado de 16 pines. Para el 74LS283, V_{CC} es el pin 16 y tierra es el pin 8, que es una configuración más estándar. Los diagramas de los pines y los símbolos lógicos de estos dos dispositivos se muestran en la Figura 6.10, en la que se indica la numeración de los pines sobre los símbolos lógicos.

es el pin 12 en el encapsulado de 16 pines. Para el 74LS283, V_{CC} es el pin 16 y tierra es el pin 8, que es una configuración más estándar. Los diagramas de los pines y los símbolos lógicos de estos dos dispositivos se muestran en la Figura 6.10, en la que se indica la numeración de los pines sobre los símbolos lógicos.

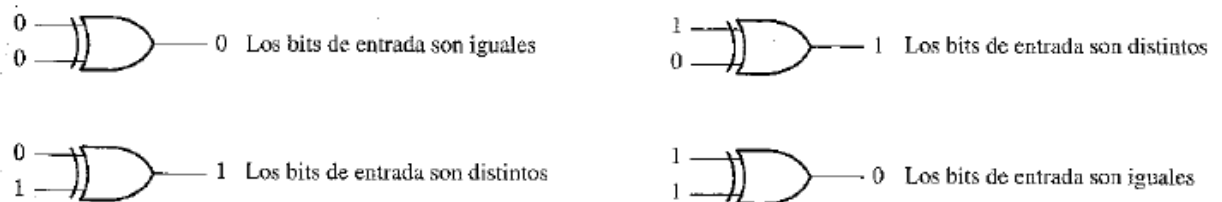


COMPARADORES

La función básica de un comparador consiste en comparar las magnitudes de dos cantidades binarias para determinar su relación. En su forma más sencilla, un circuito comparador determina si dos números son iguales.

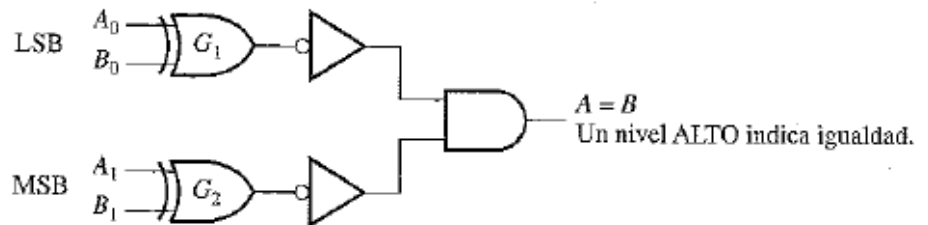
Igualdad

Como ya vimos en el Capítulo 3, la puerta OR-exclusiva se puede emplear como un comparador básico, ya que su salida es 1 si sus dos bits de entrada son diferentes y 0 si son iguales. La Figura 6.15 muestra una puerta OR-exclusiva utilizada como comparador de 2 bits.



Para comparar números binarios de dos bits, se necesita una puerta OR-exclusiva adicional. Los dos bits menos significativos (LSB) de ambos números se comparan mediante la puerta G_1 y los dos más significativos (MSB) son comparados mediante la puerta G_2 , como se muestra en la Figura 6.16. Si los dos números son iguales, sus correspondientes bits también lo son, y la salida de cada puerta OR-exclusiva será 0. Si los correspondientes conjuntos de bits no son idénticos, la salida de la puerta OR-exclusiva será un 1.

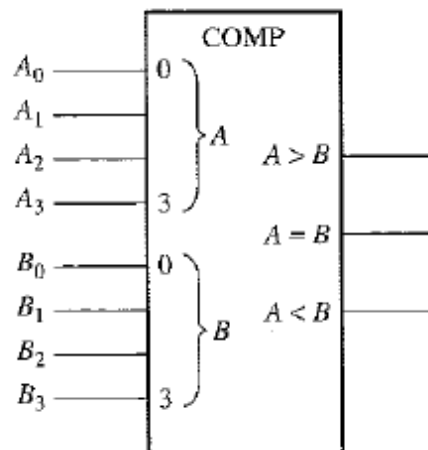
Para obtener un único resultado de salida que indique la igualdad o desigualdad entre los dos números, se pueden usar dos inversores y una puerta AND, como muestra la Figura 6.16. La salida de cada puerta OR-exclusiva se invierte y se aplica a la entrada de la puerta AND. Cuando los bits de entrada de cada OR-exclusiva son iguales, lo que quiere decir que los bits de ambos números son iguales, las entradas de la puerta AND son 1, por lo que el resultado a su salida también será 1. Cuando los dos números no son iguales, al menos uno o ambos conjuntos de bits será distinto, lo que da lugar a, al menos, un 0 en una de las entradas de la puerta AND, y el resultado a su salida será 0. Por tanto, la salida de la puerta AND indica la igualdad (1) o desigualdad (0) entre dos números.



Formato general: Número binario $A = A_1A_0$
 Número binario $B = B_1B_0$

Desigualdad

Además de disponer de una salida que indica si los dos números son iguales, muchos circuitos integrados comparadores tienen salidas adicionales que indican cuál de los dos números que se comparan es el mayor. Esto significa que existe una salida que indica cuándo el número A es mayor que el número B ($A > B$) y otra salida que indica cuándo A es menor que B ($A < B$), como se muestra en el símbolo lógico del comparador de cuatro bits de la Figura 6.18.



Para determinar una desigualdad entre los números binarios A y B , en primer lugar se examina el bit de mayor orden de cada número. Las posibles condiciones son las siguientes:

1. Si $A_3 = 1$ y $B_3 = 0$, entonces A es mayor que B .
2. Si $A_3 = 0$ y $B_3 = 1$, entonces A es menor que B .
3. Si $A_3 = B_3$, entonces tenemos que examinar los siguientes bits de orden inmediatamente inferior.

Estas tres proposiciones son válidas para cada posición que ocupen los bits dentro del número. El procedimiento general consiste en comprobar una desigualdad en cualquier posición, comenzando por los bits más significativos (MSB). Cuando se encuentra una desigualdad, la relación entre ambos números queda establecida y cualquier otra desigualdad entre bits con posiciones de orden menor debe ignorarse, ya que podrían indicar una relación entre los números completamente opuesta. *La relación de más alto orden es la que tiene prioridad.*

Un comparador de magnitud de 4 bits MSI

El 74HC85 es un comparador de tipo MSI, que también se encuentra disponible en otras familias de circuitos integrados. El diagrama de pines y el símbolo lógico se muestran en la Figura 6.20. Obsérvese que este dispositivo tiene todas las entradas y salidas del comparador visto anteriormente y, además, tiene tres entradas en cascada: $A < B$, $A = B$ y $A > B$. Estas entradas permiten utilizar varios comparadores en cascada para la comparación de cualquier número binario con más de cuatro bits. Para expandir el comparador, las salidas $A < B$, $A = B$ y $A > B$ del comparador de menor orden se conectan en cascada a las entradas del siguiente comparador de orden inmediatamente superior. El comparador de menor orden tiene que tener un nivel ALTO en la entrada $A = B$ y un nivel BAJO en las entradas $A > B$ y $A < B$.

